Microsoft Purview PCI-DSS Compliance Program

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview PCI-DSS Compliance Program 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

Why it matters . . . 6

Important boundaries . . . 6

1. Solution Overview — Microsoft Purview PCI-DSS Compliance Program 6

What this solution is . . . 6

Why this matters . . . 7

How it works . . . 7

The PCI-DSS → Purview control map (what Purview can and cannot cover) . . . 8

The role-separation model (least privilege — PCI-DSS Req 7) . . . 9

Named artifacts (used verbatim in every doc and script) . . . 9

Limitations (be honest with buyers) . . . 10

The standard deployment model . . . 10

2. Requirements Guide — Microsoft Purview PCI-DSS Compliance Program 10

2.1 Licensing — Compliance Manager premium template + advanced DLP/labeling . . . 11

2.2 PCI-DSS version, SAQ, and Cardholder Data Environment (CDE) scope decided . . . 11

2.3 Compliance role groups populated with least-privilege separation (PCI-DSS Req 7) . . . 11

2.4 Unified audit logging enabled and retained (PCI-DSS Req 10) . . . 12

2.5 Cardholder-data sensitive information type and classification confirmed (PCI-DSS Req 3) . . . 12

2.6 Cardholder-data DLP + label design agreed, to be piloted in SIMULATION (PCI-DSS Req 3) . 12

2.7 Improvement-action owners and evidence sources identified (PCI-DSS Req 12) . . . 13

2.8 Security & Compliance PowerShell + Microsoft Graph toolchain . . . 13

2.9 A pilot scope and change control (simulation before enforcement) . . . 13

Global Requirements — Microsoft Purview Solutions 14

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 14

3. Security & Compliance PowerShell + Graph toolchain . . . 15

4. Data sources & scope in scope . . . 15

5. Compliance, records, privacy & legal stakeholders . . . 15

6. A pilot scope and change control . . . 15

3. Implementation Guide — Microsoft Purview PCI-DSS Compliance Program 16

Step 1 — Baseline the score and assess readiness (Assess) . . . 16

Step 2 — Design the control map, CDE scope, and owners (Design) . . . 17

Step 3 — Assign roles with least-privilege separation (Configure — guardrails, Req 7) . . . 17
Step 4 — Enable and retain unified audit logging (Configure — Req 10) . . . 17
Step 5 — Create the PCI-DSS assessment and action plan (Configure — pilot-first) . . . 18
Step 6 — Configure the cardholder-data DLP in SIMULATION and publish the label (Configure — Req 3) . . . 18
Step 7 — Validate the evidence behind the controls (Pilot → Validate) . . . 19
Step 8 — Move DLP to enforce and expand to the full requirement set (Expand) . . . 19
5. Testing Guide — Microsoft Purview PCI-DSS Compliance Program 20
Test 1 — Role separation is in place (the access guardrail, Req 7) . . . 20
Test 2 — Licensing supports the PCI-DSS template and advanced controls . . . 20
Test 3 — Unified audit logging is enabled and retained (Req 10) . . . 20
Test 4 — The PCI-DSS assessment exists and is scored . . . 20
Test 5 — Every action has an owner, target date, status, and evidence source/link (Req 12) . . . 21
Test 6 — The cardholder-data DLP detects PAN in SIMULATION (Req 3 — no blocking) . . . 21
Test 7 — A location with no cardholder data does not false-match (precision) . . . 21
Test 8 — The cardholder-data label is published to the pilot scope (Req 3) . . . 21
Test 9 — Evidence is attached and defensible . . . 21
Test 10 — Posture report is accurate, and automation defaults are safe . . . 21
6. Troubleshooting — Microsoft Purview PCI-DSS Compliance Program 22
The PCI-DSS template isn't listed / can't be added in Compliance Manager . . . 22
The cardholder-data DLP is blocking users during the pilot . . . 22
DLP simulation shows no matches for a test card number . . . 23
The DLP false-matches on non-PAN 16-digit numbers (e.g. order IDs) . . . 23
`New-DlpCompliancePolicy` / `Get-RoleGroupMember` — "term is not recognized" . . . 23
Unified audit evidence is empty for the assessment window (Req 10) . . . 23
Compliance Score didn't move after configuring a control . . . 23
Role separation flag: someone is both administrator and contributor . . . 23
`CMPPCI-Set-Scope.ps1 -EnforceDlp -Commit` didn't enforce . . . 24
A requirement can't be implemented in Purview (firewalls, segmentation, physical, anti-malware, pen test) . . . 24
When to escalate beyond this solution . . . 24
7. Best Practices — Microsoft Purview PCI-DSS Compliance Program 24
Deliverables Summary — Microsoft Purview PCI-DSS Compliance Program 26
Documentation . . . 26
PowerShell tools (5) . . . 27
Named objects (verbatim — used across docs, scripts, and tests) . . . 28
What this solution deliberately does NOT do . . . 29

Appendix A — PowerShell Scripts 30

Executive Summary — Microsoft Purview PCI-DSS Compliance Program

The problem

If your organization stores, processes, or transmits payment-card data, **PCI-DSS is not optional** — and "are we compliant?" has no defensible answer without a control-by-control record backed by evidence. Most teams discover the gaps the hard way: in a QSA review or an SAQ, when the auditor asks *where is the cardholder data, what protects it, who logs access to it, and prove it.* Cardholder data (the Primary Account Number) tends to sprawl into email, files, and chats where no one is watching, and the evidence that controls exist is scattered across screenshots and spreadsheets. Left unaddressed, this is both a **compliance gap** and a **data-breach exposure** on the highest-value data you hold.

The solution

This solution stands up a **PCI-DSS-aligned compliance program in Microsoft Purview** — the controls that protect cardholder data, the mapping from PCI-DSS requirements to those controls, and the **audit-ready evidence** tying it all together — configured **the responsible way**:

- **The requirements Purview can actually enforce, done right.** A **DLP policy** that detects the

Credit Card Number sensitive information type and a **cardholder-data sensitivity label** (Requirement 3 — protect stored account data); **unified audit logging** enabled and retained (Requirement 10 — log and monitor access); and **least-privilege role separation** (Requirement 7 — restrict access by need-to-know).

- **Piloted in simulation first.** The cardholder-data DLP is deployed in **simulation/test mode**, so

you can see exactly what *would* be caught — and tune out false positives — **before** it can block a single legitimate payment operation. Enforcement comes only after validation, under change control.

- **The whole standard in one record. Compliance Manager**, built around the **PCI-DSS assessment

template**, maps every requirement to an owner, a status, and evidence — including** evidence links** for the requirements Purview can't enforce (firewalls, segmentation, physical, anti-malware, testing), so the record represents the complete standard.

The result is a governed, defensible position: cardholder data detected and protected, access logged, duties separated, and every control evidenced — **without over-blocking the business**.

How it works

We follow a repeatable model — **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand** — that assesses readiness, agrees the control map and CDE scope with the compliance stakeholders, configures the least-privilege guardrails and audit logging, creates the PCI-DSS assessment and action plan, deploys the cardholder-data DLP **in simulation** and the label, validates the evidence, and only then moves the DLP to enforce and expands to the full requirement set. Every high-impact change is piloted and reversible.

The solution ships as **self-guided documentation plus five PowerShell tools** (Security & Compliance PowerShell / Graph) that assess readiness and role separation, configure the cardholder-data DLP in

simulation, validate the technical evidence behind the controls, produce a leadership posture rollup, and stage role/scope and enforcement changes — all **read-only or `-WhatIf` by default**, with **no credentials in code**.

Why it matters

- **Protect the crown jewels** — detect and control cardholder data before it becomes a breach, on the highest-value data you hold.
- **Defensible evidence, clear ownership** — a QSA-ready record mapping every requirement to an owner, a status, and proof, produced as a routine rather than a fire drill.
- **No business disruption** — simulation-first DLP means you tune before you enforce, so legitimate payment operations keep running.
- **A fast, focused win** — the Purview-enforceable PCI-DSS controls delivered and validated at a fraction of consulting cost; a foundation you extend to the full requirement set.

Important boundaries

Microsoft Purview features are **licensing-dependent** — the PCI-DSS assessment template, endpoint DLP, auto-labeling, and Audit (Premium) require **Microsoft 365 E5 / E5 Compliance** or add-ons; some capabilities are in preview — and the portal is **evolving**, so availability should be **verified against your licensing and tenant**. Purview covers the **data-protection, access-governance, and logging** requirements and centralizes evidence for the rest; it does **not** provide firewalls, segmentation, vulnerability scanning, or physical controls. Critically, **how your Cardholder Data Environment is scoped, which SAQ/version applies, and whether the evidence satisfies PCI-DSS are QSA / compliance / legal determinations** — this solution provides the technical method and the evidence, not the attestation.

1. Solution Overview — Microsoft Purview PCI-DSS Compliance Program

What this solution is

This solution stands up a **PCI-DSS-aligned compliance program in Microsoft Purview**: a **control mapping** from the PCI-DSS requirements to the Purview capabilities that satisfy them, the **technical controls** themselves configured (cardholder-data DLP, a cardholder-data sensitivity label, unified audit logging, least-privilege access), and an **audit-ready evidence record** tying each control to who owns it, its status, and the proof. The spine is **Compliance Manager** (the Purview capability that turns compliance posture into a measured, points-based score with **improvement actions** and an **evidence** store), built around the **PCI-DSS assessment template** and reinforced with the data-protection controls PCI-DSS demands for stored account data.

It uses only the **Microsoft Purview portal** (`purview.microsoft.com`), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party tooling. Every high-impact control (notably the cardholder-data DLP policy) is **piloted in simulation/test mode first**, validated, and only then enforced.

> ***This delivers a Purview-backed control-and-evidence program — not a PCI attestation.*** *PCI-DSS is validated by a* ***Qualified Security Assessor (QSA)*** *or a* ***Self-Assessment Questionnaire (SAQ)*** *against your* ***Cardholder Data Environment (CDE)****.* ***Which*** *requirements and SAQ apply,* ***how*** *your CDE is scoped, and* ***whether*** *the evidence satisfies the standard are* ***QSA / compliance / legal determinations*** *— this solution provides the technical configuration method and the evidence, not the assessment verdict. It also covers only the requirements Purview can support (see the map below); network segmentation, firewalls, physical security, and anti-malware live in other systems.*

Why this matters

- **PCI-DSS obligations need mapped controls and evidence.** "Are we meeting Requirement 3 / 7 / 10?"

has no defensible answer without a control-by-control record. Compliance Manager makes it a **measured, evidenced** position so gaps are visible, owned, and tracked instead of discovered in a QSA review.

- **Cardholder data is the crown jewel.** Requirement 3 (protect stored account data) is where Purview

earns its keep: a **DLP policy** that detects the **Credit Card Number** sensitive information type, a **sensitivity label** for cardholder data, and classification insight into where Primary Account Numbers (PAN) live across the estate — configured the responsible way, **piloted in simulation** before it can block anything.

- **Evidence is the deliverable a QSA wants.** A control is only defensible with **who owns it, its

status, its test date, and the proof**. Compliance Manager centralizes that; this solution makes producing it a routine, not a fire drill.

- **Control mapping avoids duplicated effort.** One well-implemented control (unified audit logging,

say) satisfies PCI-DSS Requirement 10 **and** scores across NIST, ISO, and your internal baseline at once. The shared/control-mapping view shows that leverage so you fix once and score many.

- **CDE scoping and attestation are not technical calls.** *Which* systems are in the CDE, *which* SAQ

applies, and *whether* the evidence satisfies the standard are **QSA / compliance / legal** determinations. This solution provides the method and the evidence; it does not decide scope or sign off adequacy.

How it works

1. **Baseline and assess readiness (Assess).** Capture the current Compliance Score, confirm the

PCI-DSS assessment template is available under your licensing, snapshot the role/separation posture, and inventory the cardholder-data signals already present (audit logging, any Credit Card Number DLP/label). Run `CMPPCI-Discovery.ps1` (read-only).

1. **Design the control map and scope (Design).** With your QSA/compliance stakeholders, agree the

in-scope PCI-DSS requirements, the **CDE scope**, and the Purview-to-requirement **control map** (`PCI Control Map – Record`, `PCI Scope – Record`). CDE scoping and applicability are the stakeholders' determination.

1. **Configure the guardrails — least-privilege roles (Configure).** Assign the Purview/Compliance

role groups least-privilege and separated (administer ≠ contribute ≠ read-only) — the auditable access control behind PCI-DSS Requirement 7, snapshotted as `PCI Roles – Separation Evidence`.

1. **Create the PCI-DSS assessment and action plan (Configure).** In Compliance Manager, create

`PCI Assessment – Baseline – Pilot` from the PCI-DSS template; build the **improvement-action plan** (`PCI Action Plan – Pilot`) with an **owner, target date, status, and evidence source** per action.

1. **Configure the cardholder-data controls in SIMULATION (Configure — pilot-first).** Create the

cardholder-data DLP policy `DLP-PCI-CardholderData` (detecting the **Credit Card Number** SIT) in **test/simulation mode**, and the `Label-PCI-CardholderData` sensitivity label — so you can see what *would* be caught before anything is blocked. `CMPPCI-Configuration.ps1` is `-WhatIf` **by default**.

1. **Validate the evidence (Pilot → Validate).** Confirm the technical state behind the controls

(audit enabled, DLP simulation matching PAN, label published, roles separated) matches what the assessment claims; attach the evidence; set test status/date. `CMPPCI-Validation.ps1` (read-only) is the cross-check.

1. **Report and expand (Validate → Expand).** Roll up the posture (`PCI Posture – Report`), then move

the validated DLP policy from **simulation to enforce** under change control, extend the action plan to the full requirement set, and stand up the maintenance cadence (re-test, re-evidence, watch for template/standard updates).

The PCI-DSS → Purview control map (what Purview can and cannot cover)

PCI-DSS requirement (v4.x)	Purview capability in this program	Coverage
Req 3 — Protect stored account data	Cardholder-data **DLP** (Credit Card Number SIT), **sensitivity label**, **content/classification explorer** to locate PAN	**Direct**
Req 7 — Restrict access by need-to-know	Least-privilege **Purview role groups** + separation (evidence of RBAC governance)	**Supporting**
Req 10 — Log and monitor all access	**Unified audit logging** (Purview Audit) enabled + retained	**Direct**
Req 12 — Information-security policy	**Compliance Manager** evidence store: documented policies, owners, review dates	**Supporting**

PCI-DSS requirement (v4.x)	Purview capability in this program	Coverage
Req 9 — Physical access · Req 1/2 — Network · Req 5 — Anti-malware · Req 11 — Testing	*Not a Purview control* — tracked as **evidence links** to the owning system in Compliance Manager	**Out of scope (evidence only)**

> *Purview directly implements the **data-protection, access-governance, and logging** requirements and centralizes **evidence** for the rest. It does **not** replace firewalls, segmentation, vulnerability scanning, or physical controls. **Confirm the exact requirement set and versions against the PCI-DSS version and SAQ your acquirer/QSA requires — do not assume.***

The role-separation model (least privilege — PCI-DSS Req 7)

Role group	Who	Can (typical)
Compliance Manager Administrators / Compliance Administrator	Program owners / IT config	Manage assessments, settings, role assignment; configure DLP/labels/audit
Compliance Manager Assessors	Assessment leads	Create assessments, implement actions, set **test** status
Compliance Manager Contributors	Action owners	Edit improvement actions and evidence (not test status)
Compliance Manager Readers	QSA / oversight / auditors	**Read-only** access to the program data

> *Global Administrator only where genuinely required. **Verify the exact role-group names and rights in your tenant** — Purview role naming evolves; do not assume. Least privilege is the point.*

Named artifacts (used verbatim in every doc and script)

Every artifact this solution creates or tracks is named and reused verbatim across docs, scripts, and the portal:

Artifact	Name
PCI-DSS assessment (pilot)	`PCI Assessment – Baseline – Pilot`
PCI-DSS assessment (production)	`PCI Assessment – Production`
Cardholder-data DLP policy (simulation-first)	`DLP-PCI-CardholderData`
Cardholder-data DLP rule	`DLP-PCI-CardholderData-Rule`
Cardholder-data sensitivity label	`Label-PCI-CardholderData`
Improvement-action plan (pilot)	`PCI Action Plan – Pilot`
Evidence register	`PCI Evidence – Register`
Control-mapping record	`PCI Control Map – Record`
Posture / score report	`PCI Posture – Report`

Artifact	Name
Role-separation evidence snapshot	`PCI Roles – Separation Evidence`
Scope / CDE / owners record	`PCI Scope – Record`
Pilot scope group	`SG-PCI-Pilot`

Limitations (be honest with buyers)

• **Purview covers a subset of PCI-DSS.** It directly implements the **data-protection (Req 3)**,

access-governance (Req 7), and **logging (Req 10)** requirements and centralizes **evidence** for the rest; it does **not** provide firewalls, segmentation, vulnerability scanning, or physical controls. This is a **compliance program in Purview**, not a whole-standard implementation.

• **Premium content is licensing-gated.** The **PCI-DSS assessment template** in Compliance Manager is

a **premium template** requiring **Microsoft 365 E5 / E5 Compliance** or the **Compliance Manager Premium Assessment Add-On**; advanced DLP and auto-labeling likewise require E5-class licensing. **Verify entitlement — do not assume a template or feature is available.**

• **Compliance Manager measures; DLP/labels enforce.** Compliance Manager scores and records evidence;

it does **not** change your tenant. The cardholder-data **DLP** and **label** are the controls that actually protect data — and they are **piloted in simulation/test first** because enforcement is high-impact.

• **Attestation is not technical.** *Which* SAQ applies, *how* the CDE is scoped, and *whether* the

evidence satisfies PCI-DSS are **QSA / compliance / legal** determinations, not outputs of this tool.

• **No secrets in automation.** The provided scripts use interactive modern-auth sign-in; they never

embed credentials.

The standard deployment model

> *Baseline and assess → design the control map, CDE scope, and owners → configure least-privilege roles, create the PCI-DSS assessment and action plan, configure the cardholder-data DLP/label **in simulation** → validate the evidence behind the controls → report the posture → move DLP to enforce and expand to the full requirement set with a maintenance cadence. Pattern: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand** — CDE scoping, SAQ applicability, and attestation adequacy deferred to QSA / compliance / legal at every step.*

2. Requirements Guide — Microsoft Purview PCI-DSS Compliance Program

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant licensing, Purview roles, the PowerShell/Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, and a pilot

scope with change control) are in `global-requirements-purview.md` — this guide adds the **PCI-DSS-program-specific** detail. **Licensing, template availability, sensitive-information-type names, and portal paths are plan-dependent and the portal evolves — verify each against your own licensing and tenant. Which PCI-DSS version/SAQ applies and how your CDE is scoped are QSA/compliance determinations.**

2.1 Licensing — Compliance Manager premium template + advanced DLP/labeling

Description: Compliance Manager and the **Data Protection Baseline** are available to all Microsoft 365 subscriptions, but the **PCI-DSS assessment template** is a **premium template** requiring **Microsoft 365 E5 / E5 Compliance** or the **Compliance Manager Premium Assessment Add-On**. The cardholder-data controls likewise need the right plan: **DLP for Exchange/SharePoint/OneDrive/Teams** is broadly available, but **endpoint DLP**, **auto-labeling**, and **Audit (Premium)** long-retention require **E5 / E5 Compliance**. (GCC/GCC High and other sovereign clouds may differ.) **Why:** The PCI-DSS template and several controls are entitlement-gated; a template or feature you plan to use may not be available on your plan — the most common blocker. **Verify:** Microsoft 365 admin center → `Billing → Licenses` for E5 / E5 Compliance / add-on; Purview portal → **Compliance Manager** → **Assessment templates**, confirm **PCI-DSS** shows as available to create. `CMPPCI-Discovery.ps1` reports the licensing SKUs it can read via Graph and whether advanced DLP/audit surfaces respond. **Confirm exact entitlement with Microsoft licensing — do not assume. Implement:** Assign/purchase the required E5 / E5 Compliance / Premium Assessment Add-On before relying on the PCI-DSS template or advanced DLP/audit.

2.2 PCI-DSS version, SAQ, and Cardholder Data Environment (CDE) scope decided

Description: The **PCI-DSS version** you must meet (the current standard is **v4.0 / v4.0.1** — confirm the version your acquirer/QSA requires), the applicable **SAQ** (or full ROC), and the **Cardholder Data Environment (CDE)** scope — the systems, people, and processes that store, process, or transmit **cardholder data (PAN)** — plus which requirements are **in scope** for Purview to support versus tracked as **evidence links** to another system. **Why:** CDE scoping and SAQ applicability are the **QSA/compliance determination** that defines the whole program; the score and controls are only meaningful against a defined scope. Getting scope wrong either over-builds or leaves the CDE unprotected. **Verify:** The version, SAQ, in-scope requirements, and CDE boundary are recorded and confirmed by the QSA/compliance owners (`PCI Scope – Record`, `PCI Control Map – Record`). **Implement:** Engage the QSA/compliance function; capture the version, SAQ, CDE boundary, and the Purview-to-requirement control map before configuring.

2.3 Compliance role groups populated with least-privilege separation (PCI-DSS Req 7)

Description: The Compliance/Compliance-Manager role groups — **Compliance Manager Administrators**, **Assessors**, **Contributors**, **Readers** (plus **Compliance Administrator** / **Compliance Data Administrator** for DLP/label/audit configuration) — populated so administrators are separated from action **contributors** and from read-only **readers/auditors/QSA**. **Why:** Least-privilege separation *is* PCI-DSS Requirement 7 (restrict access by need-to-know) applied to the compliance program itself, and it is the auditable access control behind the evidence. Evidence is only defensible if access to it was

governed. **Verify:** Purview portal → `Settings → Roles & scopes → Permissions` (or, via `Connect-IPPSSession`, `Get-RoleGroupMember 'Compliance Manager Contributors'` etc.) shows the intended, separated membership. `CMPPCI-Discovery.ps1` prints each group's membership as `PCI Roles – Separation Evidence` and flags administrators who are also contributors. **Verify the exact role-group names in your tenant — Purview role naming evolves. Implement:** Assign membership under change control with `CMPPCI-Set-Scope.ps1` (`-WhatIf` first, then `-Commit`); grant the **minimum** role each person needs and keep readers/auditors/QSA read-only.

2.4 Unified audit logging enabled and retained (PCI-DSS Req 10)

Description: Unified audit logging (Purview Audit) **enabled** tenant-wide, with a **retention** period that meets PCI-DSS Requirement 10's log-retention expectation (PCI-DSS calls for a minimum retention with recent logs immediately available — **confirm the exact figure against your PCI-DSS version/QSA**). **Audit (Premium)** provides longer default retention and custom retention policies and requires E5-class licensing. **Why:** Requirement 10 (log and monitor all access to system components and cardholder data) is a **direct** Purview control — the audit log is the evidence that access is recorded. If logging is off or under-retained, the requirement fails at the source. **Verify:** `Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled` returns `True`; check audit-retention policies in the portal. `CMPPCI-Discovery.ps1` / `CMPPCI-Validation.ps1` report the ingestion state and any custom retention policies they can read. **Confirm the required retention figure against your PCI-DSS version/QSA — do not assume. Implement:** Enable unified audit logging (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`) and configure an audit-retention policy that meets the requirement; verify Audit (Premium) entitlement for extended retention.

2.5 Cardholder-data sensitive information type and classification confirmed (PCI-DSS Req 3)

Description: The **sensitive information type (SIT)** that detects cardholder data — the built-in **Credit Card Number** SIT (and any custom/**EDM** SIT your CDE needs for stored PAN) — confirmed available, and an understanding of **where cardholder data lives** across the in-scope workloads (Exchange, SharePoint, OneDrive, Teams, and endpoints where in scope). **Why:** Requirement 3 (protect stored account data) keys off detecting PAN. The DLP policy and label depend on the SIT; if the SIT is wrong or PAN lives outside the scanned workloads, protection has gaps. **Verify:** `Get-DlpSensitiveInformationType` lists the **Credit Card Number** SIT (and any custom SITs); Purview **Data Classification / Content Explorer** shows where the SIT is detected. `CMPPCI-Discovery.ps1` confirms the SIT is present. **Note:** PAN detection is pattern/checksum-based and can false-positive/negative — treat matches as signals to review, not proof. **Implement:** Confirm the built-in SIT; where the CDE needs precise matching of known PANs, plan a custom **Exact Data Match (EDM)** SIT (its own effort). Confirm the in-scope workloads cover where PAN resides.

2.6 Cardholder-data DLP + label design agreed, to be piloted in SIMULATION (PCI-DSS Req 3)

Description: The design of the cardholder-data **DLP policy** (`DLP-PCI-CardholderData`) and **sensitivity label** (`Label-PCI-CardholderData`) — the workloads in scope, the actions (audit / notify / restrict / block), and the exceptions — with the explicit intent to deploy the DLP policy in **test/simulation**

mode first and only enforce after validation. **Why:** DLP and labels are **high-impact**: an over-broad block on cardholder data can disrupt legitimate payment operations. Simulation mode shows what *would* be caught with no user impact, so you tune before enforcing. This is the category's Assess→…→Pilot(simulate)→Expand(enforce) rule. **Verify:** The policy/label design, in-scope workloads, actions, and exceptions are documented, and the agreed rollout is **simulation** → **validate** → **enforce**. `CMPPCI-Discovery.ps1` reports any existing Credit Card Number DLP policy/label and its mode. **Implement:** Build the DLP policy in `TestWithNotifications`/`TestWithoutNotifications` **(simulation) mode** with `CMPPCI-Configuration.ps1` (`-WhatIf` first, then `-Commit`); publish the label to the pilot scope; move to enforce only after Testing (`05-testing.md`) passes.

2.7 Improvement-action owners and evidence sources identified (PCI-DSS Req 12)

Description: For the pilot assessment, the **owner** of each improvement action, the **target date**, and the **evidence source** — including, for the out-of-scope-for-Purview requirements (firewalls, segmentation, physical, anti-malware, testing), the **link to the owning system's evidence** so the whole standard is represented in one record. **Why:** A score with no owners and no evidence is not defensible; Requirement 12 expects documented, owned security policy. Ownership and evidence source turn recommendations into a trackable, auditable plan (`PCI Action Plan – Pilot`, `PCI Evidence – Register`). **Verify:** Each in-scope action has an owner, a target date, and a named evidence source; out-of-scope requirements carry an evidence link. **Implement:** Build the action plan with `CMPPCI-Configuration.ps1` (`-WhatIf` first) from the owner-provided list; capture the evidence source per action.

2.8 Security & Compliance PowerShell + Microsoft Graph toolchain

Description: A workstation with the **ExchangeOnlineManagement** module (`Connect-IPPSSession` for role-group, DLP, label, and audit cmdlets) and **Microsoft Graph PowerShell** (for licensing evidence), used with a modern-auth / MFA-capable admin holding the appropriate Compliance and read roles. **Why:** The five provided scripts assess readiness, configure the cardholder-data DLP in simulation, validate the technical evidence, and report — through Security & Compliance PowerShell and Graph. Compliance Manager assessments/evidence are **portal-managed**; the scripts support them. **Verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed; `Get-RoleGroup`, `Get-DlpCompliancePolicy`, and `Get-AdminAuditLogConfig` return data. `CMPPCI-Discovery.ps1` performs these checks. **Implement:** `Install-Module ExchangeOnlineManagement`; `Install-Module Microsoft.Graph`; sign in with the role-holding admin and grant the read scopes the scripts request.

2.9 A pilot scope and change control (simulation before enforcement)

Description: A named **pilot scope** (`SG-PCI-Pilot` — users/sites in the CDE or a representative subset) validated first — the assessment, action plan, DLP **in simulation**, and label — before tenant-wide enforcement, plus a **change-control** path for moving DLP from simulation to enforce (consistent with `global-requirements-purview.md` §6). **Why:** Enforcing cardholder-data DLP across the whole tenant at once is high-impact; validating one scope end-to-end proves ownership, evidence, and the DLP's precision before scaling. DLP enforcement is piloted in simulation under change control. **Verify:** A named pilot scope and owner exist; the DLP policy is in simulation mode; a change window/approval path for

enforcement is defined. **Implement:** Choose the pilot scope, validate it end-to-end (Testing), then expand; agree change control before moving DLP to enforce.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview PCI-DSS Compliance Program

Follow these steps **in order**. Configuration is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`) and, where automated, via **Security & Compliance PowerShell** (`Connect-IPPSSession`) and **Microsoft Graph**. Artifact names follow the Overview's naming and are reused verbatim. **Portal paths, template/SIT availability, and cmdlets are plan-dependent and the portal evolves — verify against your tenant. The cardholder-data DLP policy is deployed in SIMULATION/TEST mode first and enforced only after validation.**

> ***Model:*** *Assess readiness, design the control map + CDE scope + owners, configure least-privilege roles + the PCI-DSS assessment + the cardholder-data DLP (in simulation) + the label, validate the evidence, report, then move DLP to enforce and expand. Pattern:* ***Assess → Design → Configure → Pilot (simulate/test) → Validate → Expand. Compliance Manager measures; DLP/labels enforce.*** *CDE scoping, SAQ applicability, and attestation adequacy are* ***QSA/compliance/legal determinations****, not decisions this guide makes.*

Step 1 — Baseline the score and assess readiness (Assess)

1. Confirm licensing (Requirement 2.1): portal → **Compliance Manager** → **Assessment templates**, the

PCI-DSS template shows as available to create. **Verify entitlement with Microsoft licensing — do not assume.**

1. Run `CMPPCI-Discovery.ps1` (read-only). It reports: the licensing SKUs it can read; the membership

of each Compliance role group (the `PCI Roles – Separation Evidence` snapshot) and any admin who is also a contributor; whether **unified audit logging** is enabled (Req 10); whether the **Credit Card Number** SIT is present (Req 3); and any existing Credit Card Number **DLP policy/label** and its mode. Note the current **Compliance Score** from the portal dashboard as your starting point.

1. **Pass/fail gate:** licensing confirmed for the PCI-DSS template, the role groups exist, audit

logging state and the Credit Card Number SIT are known, and the baseline score is recorded. **Do not proceed** until the QSA/compliance owners (Requirement 2.2) have confirmed the PCI-DSS version, SAQ, and CDE scope. Fail → resolve before configuring.

Step 2 — Design the control map, CDE scope, and owners (Design)

1. With the QSA/compliance owners, confirm the **PCI-DSS version/SAQ**, the **CDE boundary**, and the **in-scope requirements** (Requirement 2.2). CDE scoping and applicability are their determination.

1. Build the **Purview-to-requirement control map** (`PCI Control Map – Record`): Req 3 → cardholder DLP + label + classification; Req 7 → least-privilege roles; Req 10 → unified audit logging; Req 12 → Compliance Manager evidence store; the remaining requirements (firewalls, segmentation, physical, anti-malware, testing) → **evidence links** to their owning systems.

1. Assign a **business owner** per requirement/action and record the scope as `PCI Scope – Record` (version, SAQ, CDE boundary, in-/out-of-scope requirements, owners).

Step 3 — Assign roles with least-privilege separation (Configure — guardrails, Req 7)

1. Portal → `Settings → Roles & scopes → Permissions`: populate the role groups (Requirement 2.3):

• **Compliance Manager Administrators / Compliance Administrator** → program owners / IT config (also configure DLP/labels/audit).

• **Compliance Manager Assessors** → assessment leads who create assessments and set **test** status.

• **Compliance Manager Contributors** → improvement-action **owners** who update actions/evidence.

• **Compliance Manager Readers** → QSA / oversight / auditors, **read-only**.

1. Or manage membership with `CMPPCI-Set-Scope.ps1` — it composes `Add-RoleGroupMember` / `Remove-RoleGroupMember`, **runs with `-WhatIf` by default**, and prints exactly what it would change; add `-Commit` to apply. Grant the **minimum** role each person needs.

1. **Pass/fail gate:** each person holds the least-privilege role; administrators are separated from contributors; readers/auditors/QSA are read-only. Re-run `CMPPCI-Discovery.ps1` and confirm the `PCI Roles – Separation Evidence` snapshot. **Rollback:** remove any over-granted membership with `CMPPCI-Set-Scope.ps1 -Commit`.

Step 4 — Enable and retain unified audit logging (Configure — Req 10)

1. Confirm **unified audit logging** is on: `Get-AdminAuditLogConfig | Select UnifiedAuditLogIngestionEnabled → True. If not, enable it: Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true` (allow time to take effect).

1. Configure an **audit-retention policy** that meets your PCI-DSS version's Requirement 10 retention (Audit (Premium) enables longer/custom retention; verify entitlement — Requirement 2.1/2.4).

1. **Pass/fail gate:** audit ingestion is `True` and a retention policy meeting the requirement exists.

`CMPPCI-Validation.ps1` confirms the state. Fail → enable logging / add the retention policy before claiming Req 10 evidence. **Rollback:** audit configuration changes are reversible in the portal / via `Set-AdminAuditLogConfig`, but keep logging **on** — disabling it fails Req 10.

Step 5 — Create the PCI-DSS assessment and action plan (Configure — pilot-first)

1. Portal → **Compliance Manager** → **Assessments** → **Add assessment**: create

`PCI Assessment – Baseline – Pilot` from the **PCI-DSS** template, in the agreed group. (The premium template requires the entitlement from Requirement 2.1.)

1. Build the **improvement-action plan** (`PCI Action Plan – Pilot`): assign each action an **owner**,

target date, and **implementation status** in the portal; for out-of-scope-for-Purview requirements, record the **evidence link** to the owning system.

1. Use `CMPPCI-Configuration.ps1` (`-WhatIf` default; `-Commit` writes local tracking artifacts) to

validate the plan and write `PCI Action Plan – Pilot` / `PCI Evidence – Register` from the owner-provided list. Compliance Manager has no broad write API, so **assignments and evidence upload happen in the portal**, driven by these artifacts.

1. **Pass/fail gate:** the pilot assessment exists with an owner and a visible score, and every action

has an owner, target date, status, and evidence source/link. Fail → complete the plan before configuring the data controls. **Rollback:** an assessment can be removed in the portal without affecting tenant configuration.

Step 6 — Configure the cardholder-data DLP in SIMULATION and publish the label (Configure — Req 3)

1. **Create the DLP policy in simulation/test mode** — this is high-impact, so it starts non-enforcing.

Use `CMPPCI-Configuration.ps1` (`-WhatIf` default): with `-Commit` it creates `DLP-PCI-CardholderData` via `New-DlpCompliancePolicy -Mode TestWithNotifications` (simulation) across the in-scope workloads and the rule `DLP-PCI-CardholderData-Rule` via `New-DlpComplianceRule` matching the **Credit Card Number** SIT (Requirement 2.5/2.6). **Simulation mode audits/notifies but does not block.**

1. **Publish the sensitivity label** `Label-PCI-CardholderData` for cardholder data (portal →

Information Protection → **Labels**) and a label policy scoped to `SG-PCI-Pilot`; where entitled, plan **auto-labeling** on the Credit Card Number SIT (its own pilot). Label authoring is portal-managed; the script reports label presence.

1. **Pass/fail gate:** `DLP-PCI-CardholderData` exists in **simulation mode** with the Credit Card

Number rule, and `Label-PCI-CardholderData` is published to the pilot scope. Confirm with `CMPPCI-Validation.ps1`. Fail → correct the policy/rule/label before validating. **Rollback:** the DLP policy is non-enforcing in simulation; it can be disabled/removed with `Set-DlpCompliancePolicy` / `Remove-DlpCompliancePolicy` with no user impact.

Step 7 — Validate the evidence behind the controls (Pilot → Validate)

1. Run the Testing Guide (`05-testing.md`). Confirm: the pilot assessment shows a score; each action's

implementation and test status is set with **evidence**; audit logging is on and retained (Req 10); the **DLP simulation is matching** test PAN in the pilot scope (Req 3) without blocking; the label is published; role separation holds (Req 7). `CMPPCI-Validation.ps1` (read-only) is the cross-check.

1. **Review the DLP simulation results** in the portal (policy → **Simulation/Overview**): confirm it

catches genuine cardholder data and tune out false positives **before** enforcing. Do **not** mark Req 3 implemented on simulation alone — note it as *piloted, pending enforcement*.

1. Run `CMPPCI-Report.ps1` for the `PCI Posture – Report` (score inputs, control-to-evidence map, audit

state, DLP mode + simulation summary, role separation, licensing).

1. **Pass/fail gate:** the pilot assessment is owned, evidenced, and tested; audit logging validated;

DLP simulation validated and tuned; label published; posture report accurate. Fail → complete evidence/testing and re-validate.

Step 8 — Move DLP to enforce and expand to the full requirement set (Expand)

1. **Only after Step 7 passes**, move `DLP-PCI-CardholderData` from simulation to **enforce** under

change control: `CMPPCI-Set-Scope.ps1` (or `Set-DlpCompliancePolicy -Mode Enable`) with `-WhatIf` first, then `-Commit`, and expand the policy scope beyond `SG-PCI-Pilot` as agreed. Watch the first enforcement window for legitimate-workflow impact.

1. With the QSA/compliance owners, create `PCI Assessment – Production` (or extend the pilot to the full

requirement set) and complete the action plan/evidence for every in-scope requirement, leaning on **control mapping** so shared controls (audit, access) score across frameworks — the incremental effort is the *gap*.

1. Stand up the **maintenance cadence**: periodic re-test/re-evidence, review of role membership

(joiners/leavers), watch for **PCI-DSS version updates** and Microsoft **template updates**, re-run `CMPPCI-Validation.ps1` and `CMPPCI-Report.ps1` on a schedule for the posture trend.

1. **Pass/fail gate:** the DLP is enforcing without disrupting legitimate payment operations, the full

in-scope set is owned and evidenced, audit logging is retained, and a recurring review is scheduled. **Rollback:** revert the DLP to simulation with `Set-DlpCompliancePolicy -Mode TestWithNotifications` if enforcement causes unacceptable impact; any assessment can be removed in the portal.

What this guide does NOT decide

- **How the CDE is scoped, which SAQ/version applies, and whether the evidence satisfies PCI-DSS** — a

QSA / compliance / legal determination. This guide configures the controls, the assessment, the action plan, and the evidence, and produces the score and posture; scope and attestation belong to those functions.

- **The requirements Purview cannot implement** — firewalls, network segmentation, physical security, anti-malware, and vulnerability/penetration testing are **separate systems**; here they are tracked as **evidence links** in Compliance Manager, not configured.

5. Testing Guide — Microsoft Purview PCI-DSS Compliance Program

Run these after assigning roles (Step 3), enabling audit logging (Step 4), and creating the pilot assessment, action plan, cardholder-data DLP **in simulation**, and label (Steps 5–6), and again after each change and before enforcing/expanding. Every test here is **safe and read-only** or contained to **simulation/pilot scope** — the DLP policy is non-enforcing while you test, and Compliance Manager changes nothing on the tenant. The point is to prove the **score, ownership, evidence, audit logging, DLP detection, label, and role separation** all work before enforcement. **Whether the resulting evidence satisfies PCI-DSS is a QSA/compliance determination — these tests validate the mechanism.**

Test 1 — Role separation is in place (the access guardrail, Req 7)

Do: Run `CMPPCI-Discovery.ps1`; review the `PCI Roles – Separation Evidence` snapshot of each role group's membership. **Expected:** Administrators are separated from contributors; assessors set test status; readers/auditors/QSA are **read-only**. No administrator is silently also a contributor unless the governance owners accepted it. **Fail** → correct membership with `CMPPCI-Set-Scope.ps1` (Implementation Step 3) before proceeding.

Test 2 — Licensing supports the PCI-DSS template and advanced controls

Do: Run `CMPPCI-Discovery.ps1` and, in the portal → **Compliance Manager** → **Assessment templates**, confirm **PCI-DSS** is available to create. **Expected:** The PCI-DSS premium template is available under your entitlement, and advanced DLP/audit surfaces respond. **Fail** → resolve licensing (Requirement 2.1) before creating the assessment or relying on advanced controls.

Test 3 — Unified audit logging is enabled and retained (Req 10)

Do: Run `CMPPCI-Validation.ps1` (read-only). **Expected:** `UnifiedAuditLogIngestionEnabled` is `True` and an audit-retention policy meeting your PCI-DSS version's Requirement 10 is present. **Fail** → enable logging / add the retention policy (Implementation Step 4); Req 10 evidence is not defensible without it.

Test 4 — The PCI-DSS assessment exists and is scored

Do: In the portal → **Compliance Manager** → **Assessments**, open `PCI Assessment – Baseline – Pilot`. **Expected:** The assessment is in the agreed group, has an owner, and shows a **Compliance Score** and its improvement actions mapped to PCI-DSS requirements. **Fail** → recreate/repair the

assessment (Implementation Step 5).

Test 5 — Every action has an owner, target date, status, and evidence source/link (Req 12)

Do: Run `CMPPCI-Report.ps1`; review the `PCI Action Plan – Pilot` / `PCI Evidence – Register` rollup against the assessment's actions in the portal. **Expected:** Each in-scope action has an **owner**, **target date**, **implementation status**, and **named evidence source**; out-of-scope-for-Purview requirements carry an **evidence link** to the owning system; nothing is unassigned. **Fail** → complete the plan with `CMPPCI-Configuration.ps1` and assign in the portal.

Test 6 — The cardholder-data DLP detects PAN in SIMULATION (Req 3 — no blocking)

Do: Confirm `DLP-PCI-CardholderData` is in **simulation/test mode** (`CMPPCI-Validation.ps1`), then place a **synthetic test card number** (a standard test PAN, never a real card) in a pilot-scope location and review the policy → **Simulation/Overview** in the portal. **Expected:** The policy **matches** the test PAN via the Credit Card Number rule and records the match **without blocking** the user — simulation audits/notifies only. **Fail** → check the rule's SIT, confidence, and scope; the DLP isn't detecting cardholder data yet.

Test 7 — A location with no cardholder data does not false-match (precision)

Do: Review the simulation results for content that contains card-like but non-PAN numbers (e.g. a 16-digit order ID) in the pilot scope. **Expected:** The policy does **not** flag non-PAN numbers (the Credit Card Number SIT uses a Luhn checksum + context), or false positives are few enough to tune out before enforcing. **Fail** → raise the rule's confidence/instance threshold or add context requirements before moving to enforce.

Test 8 — The cardholder-data label is published to the pilot scope (Req 3)

Do: Confirm `Label-PCI-CardholderData` is published (`CMPPCI-Validation.ps1` reports label presence; verify the label policy scope in the portal). **Expected:** The label is available to `SG-PCI-Pilot` users (and, where entitled, auto-labeling on the Credit Card Number SIT is configured as a separate pilot). **Fail** → publish/scope the label (Implementation Step 6).

Test 9 — Evidence is attached and defensible

Do: Open two or three in-scope actions in the portal; confirm the uploaded **evidence**, **owner**, and **test date** are present, and that `PCI Evidence – Register` records the source. **Expected:** Each action carries proof, an owner, and a date — a QSA-ready record. Req 3 is recorded as *piloted, pending enforcement* until the DLP is enforcing. **Fail** → attach the evidence and record it in the register.

Test 10 — Posture report is accurate, and automation defaults are safe

Do: Run `CMPPCI-Report.ps1` for the `PCI Posture – Report`; then run `CMPPCI-Configuration.ps1` and `CMPPCI-Set-Scope.ps1` with no action switch. **Expected:** The report rolls up the score inputs, the control-to-evidence map, audit state, DLP mode + simulation summary, role separation, and licensing — matching the portal, and noting the **portal Compliance Manager export** as authoritative. `CMPPCI-Configuration.ps1` and `CMPPCI-Set-Scope.ps1` run in `-WhatIf` mode and only *describe* changes; `CMPPCI-Discovery.ps1`, `CMPPCI-Validation.ps1`, and `CMPPCI-Report.ps1` are read-only. **Fail** → stop; do not run with `-Commit`, and do not enforce DLP, until the dry-run output and simulation results are correct.

Gate to enforce + expand: Tests 1–2 (guardrails + licensing), 3 (audit/Req 10), 4–5 (assessment + ownership), 6–8 (DLP detection in simulation + precision + label/Req 3), and 9–10 (evidence + reporting + safety) all pass for the pilot → move the DLP from simulation to **enforce** under change control and expand to the full requirement set (Implementation Step 8). Any fail → correct and re-test. **Whether the score/evidence satisfies PCI-DSS, and how the CDE is scoped, are QSA/compliance determinations, not technical ones.**

6. Troubleshooting — Microsoft Purview PCI-DSS Compliance Program

The failure modes you'll hit building and maintaining a PCI-DSS program in Purview. Each: **symptom** → **likely cause** → **fix.** Most "Req 3 isn't covered" reports are actually licensing, a DLP policy in the wrong mode, audit that was never on, or evidence that was never attached — this guide separates those from real faults. Automation is **Security & Compliance PowerShell** (`Connect-IPPSSession`) plus **Exchange Online PowerShell** (`Connect-ExchangeOnline`) for the audit config.

The PCI-DSS template isn't listed / can't be added in Compliance Manager

- **Cause A — licensing.** PCI-DSS is a **premium template** requiring **E5 / E5 Compliance** (premium templates may be metered). **Fix:** confirm the plan (`CMPPCI-Discovery.ps1`); assign E5 / E5 Compliance; check premium-template availability — **verify entitlement with Microsoft licensing, do not assume** (Requirement 2.1).
- **Cause B — role.** You lack **Compliance Manager Administrator**. **Fix:** assign the role group (Requirement 2.3). Template names/versions change — **verify against current Microsoft documentation.**

The cardholder-data DLP is blocking users during the pilot

- **Cause:** the policy was created or flipped to **Enable** (enforce) instead of simulation. During the pilot it must be `TestWithNotifications` (simulation — audits/notifies, does not block).
- **Fix:** confirm `Get-DlpCompliancePolicy "DLP-PCI-CardholderData" | Select Mode`; roll it back with `CMPPCI-Set-Scope.ps1 -SimulateDlp -PolicyName "DLP-PCI-CardholderData" -Commit`. Enforce only after 05-testing.md passes, deliberately (`-EnforceDlp -Commit -ConfirmEnforce`).

DLP simulation shows no matches for a test card number

- **Cause A — propagation latency.** New/changed DLP policies take time to propagate before they

evaluate content. **Fix:** allow time, retest.

- **Cause B — SIT/scope mismatch.** The rule's **Credit Card Number** SIT, confidence, or location scope

doesn't cover the pilot content. **Fix:** confirm the rule's SIT name matches your tenant, the confidence isn't set too high, and the pilot location is in scope (Requirement 2.5/2.6). Always test with a **synthetic test PAN**, never a real card.

The DLP false-matches on non-PAN 16-digit numbers (e.g. order IDs)

- **Cause:** confidence/context too loose. The Credit Card Number SIT uses a **Luhn checksum + context**;

low confidence lets card-like numbers through. **Fix:** raise the rule's confidence or instance threshold, or add context requirements, and re-review the simulation **before** enforcing (Test 7).

`New-DlpCompliancePolicy` / `Get-RoleGroupMember` — "term is not recognized"

- **Cause:** wrong endpoint. DLP, label, and role-group cmdlets are **S&C PowerShell**

(`Connect-IPPSSession`); `Get/Set-AdminAuditLogConfig` is **Exchange Online** (`Connect-ExchangeOnline`). **Fix:** connect the right endpoint; both ship in `ExchangeOnlineManagement`.

Unified audit evidence is empty for the assessment window (Req 10)

- **Cause:** unified audit logging was enabled **after** the activity, or wasn't on — **audit is not

retroactive**. Fix: **enable it now (**`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true`**); evidence accrues from enablement forward. Add an** audit-retention policy** meeting your version's Req 10 retention (Audit (Premium) enables longer retention — verify entitlement). Capture the gap honestly rather than presenting it as compliant.

Compliance Score didn't move after configuring a control

- **Cause:** the improvement action's **implementation/test status and evidence weren't updated** in the

portal. Compliance Manager scores off recorded improvement-action status, not raw tenant state.

- **Fix:** attach the artifact and set the action's status in the portal (Requirement 2.2/Req 12);

scores update on the portal's cadence. `CMPPCI-Report.ps1` shows the tenant control state, but the **portal export is the authoritative score.**

Role separation flag: someone is both administrator and contributor

- **Cause:** an identity holds two role groups that PCI-DSS Req 7 wants separated. **Fix:** if the

overlap wasn't accepted by the governance owners, remove the excess membership with `CMPPCI-Set-Scope.ps1 -RoleGroup "…" -Remove user@contoso.com -Commit`. Grant the **minimum** role; keep QSA/auditors **read-only**.

`CMPPCI-Set-Scope.ps1 -EnforceDlp -Commit` didn't enforce

- **Cause:** by design — enforcement is **double-gated**. `-Commit` alone is refused; you must also pass

`-ConfirmEnforce`. **Fix:** re-run with `-EnforceDlp -Commit -ConfirmEnforce`, and only after all 05-testing.md gates pass. This prevents an accidental enforce bundled with a membership edit.

A requirement can't be implemented in Purview (firewalls, segmentation, physical, anti-malware, pen test)

- **Cause:** those controls live in **other systems** — Purview has no configuration for them. **Fix:**

in `PCI Control Map – Record`, record them as **evidence links** to the owning system, not as Purview controls; attach their evidence to the Compliance Manager action. Don't force Purview to "cover" them.

When to escalate beyond this solution

- **How the CDE is scoped, which SAQ/version applies, and whether the evidence satisfies PCI-DSS** →

QSA / compliance / legal (this solution configures controls, the assessment, and evidence, and produces the posture; scope and attestation are theirs — deferred).

- **Whether a false-positive exception is acceptable in the payment workflow** → the **data/payment

owner** with compliance.

- **A capability genuinely missing despite correct licensing** → verify against current Microsoft

documentation and open a Microsoft support case; Compliance Manager templates and Purview capabilities evolve.

7. Best Practices — Microsoft Purview PCI-DSS Compliance Program

- **Let the QSA/compliance owners scope the CDE — not the technical team.** *Which* systems are in the

Cardholder Data Environment, *which* SAQ/version applies, and *whether* the evidence satisfies the standard are **QSA/compliance/legal determinations**. The tool provides method, score, and evidence — not the attestation. Record the scope decision in `PCI Scope – Record` and configure to it.

- **Pilot cardholder-data DLP in simulation, always.** Deploy `DLP-PCI-CardholderData` in

`TestWithNotifications`/`TestWithoutNotifications` **(simulation)** first. It audits and notifies but never blocks, so you see what *would* be caught with zero user impact. Only move to **enforce** after the

simulation is validated and tuned — enforcing a card-number block blind can break legitimate payment operations.

- **Tune for precision before you enforce.** The Credit Card Number SIT uses a Luhn checksum plus context, but 16-digit order IDs and test data still cause noise. Raise the rule's confidence/instance threshold and add context requirements until the simulation flags real PAN and little else — then enforce.

- **Enable and retain the audit log — it *is* Requirement 10.** Keep unified audit logging **on** and retained to your PCI-DSS version's figure (Audit (Premium) for extended/custom retention). Disabling it fails Req 10 at the source; `CMPPCI-Validation.ps1` is your on-going check.

- **Least privilege and separation are Requirement 7 applied to yourselves.** Keep **Administrators** separate from **Contributors**, and keep **Readers/auditors/QSA** read-only. Manage membership with `CMPPCI-Set-Scope.ps1` (`-WhatIf` then `-Commit`) and re-snapshot with `CMPPCI-Discovery.ps1` — membership drifts as people join and leave.

- **Evidence is the deliverable a QSA wants.** Attach documentation, test results, an owner, and a test date to each action. The score is only defensible with the proof behind it — `PCI Evidence – Register` is the audit-ready record. Record Requirement 3 as *piloted, pending enforcement* until the DLP is actually enforcing.

- **Represent the whole standard, even what Purview can't enforce.** Firewalls, segmentation, physical security, anti-malware, and vulnerability/penetration testing live in other systems — track them as **evidence links** in Compliance Manager so `PCI Assessment` reflects the complete requirement set, not just the Purview slice.

- **Lean on control mapping.** Implement a shared control once (unified audit logging, least-privilege access) and let it score across PCI-DSS, NIST, ISO, and your internal baseline. Prioritize **high-point, widely-mapped** actions, and when you add a framework only do the *gap*.

- **Never mark a control implemented without proof.** A false "implemented" inflates the score and fails the first QSA review. If the DLP is only in simulation, Req 3 is not "implemented"; if audit logging is off, Req 10 is not met — `CMPPCI-Validation.ps1` is your cross-check against wishful status.

- **Always dry-run automation.** `CMPPCI-Configuration.ps1` and `CMPPCI-Set-Scope.ps1` default to `-WhatIf`. Read every line of the planned change — especially the DLP policy definition and any mode change from simulation to enforce — and run `-Commit` only after the output is right. The discovery, validation, and report scripts are read-only.

- **Never store or log real PAN in test.** Use standard **synthetic test card numbers** for DLP validation, never a real cardholder's account number, and keep test data out of scripts and logs. Treat DLP matches as signals to review, not proof — detection is pattern-based and can err.

- **Watch for PCI-DSS version and template updates.** PCI-DSS evolves (v4.x introduced new

requirements); Microsoft updates the PCI-DSS assessment template accordingly. A template/version update can add controls and move your score — review on a cadence and re-baseline affected assessments.

- **Verify licensing and availability against your tenant.** The PCI-DSS template, endpoint DLP, auto-labeling, and Audit (Premium) are entitlement-gated; GCC/GCC High may differ. Confirm — never assume a template or feature is available.
- **The portal is authoritative.** Assessments, evidence upload, DLP simulation results, and the official score/export live in the Purview portal; the scripts assess readiness, configure the DLP in simulation, validate technical evidence, and report — they support the portal, they don't replace it.

Deliverables Summary — Microsoft Purview PCI-DSS Compliance Program

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Solution Overview (`01-solution-overview.md`)	What the program delivers, the PCI-DSS → Purview control map, the role-separation model, the named artifacts, and the Assess→Design→Configure→Pilot(simulate)→Validate→Expand model.
Requirements (`02-requirements.md`)	Solution-specific prerequisites — licensing (PCI-DSS premium template + advanced DLP/audit), PCI-DSS version/SAQ/CDE scope, least-privilege role separation, unified audit logging, the Credit Card Number SIT, the DLP/label design (simulation-first), action owners/evidence, toolchain, pilot scope + change control.
Implementation (`03-implementation.md`)	Ordered steps: assess, design the control map/scope, configure roles + audit logging + the PCI-DSS assessment + the cardholder-data DLP **in simulation** + the label, validate, then enforce and expand — with pass/fail gates and rollback.
Testing & Validation (`05-testing.md`)	Ten pass/fail gates from role separation, licensing, and audit logging through DLP detection-in-simulation, precision, label, evidence, and reporting/safety — the gate to enforce and expand.
Troubleshooting (`06-troubleshooting.md`)	The common failure modes (template not available, DLP not matching / over-matching, audit logging off, label not appearing, role/permission issues, false PAN matches) and how to resolve them.

Document	Purpose
Best Practices (`07-best-practices.md`)	QSA-scopes-the-CDE, simulation-first-DLP, tune-for-precision, audit-is-Req-10, least-privilege, evidence-is-the-deliverable, control-mapping, never-store-real-PAN, and version/template-update guidance.
Executive Summary (`executive-summary.md`)	One-page business framing for decision-makers.

PowerShell tools (5)

All tools are **read-only or `-WhatIf` by default**, authenticate **interactively** (no credentials in code), and use **Security & Compliance PowerShell** (`Connect-IPPSSession`) and/or **Microsoft Graph** (`Connect-MgGraph`). Compliance Manager assessment/evidence *authoring* is a portal action; these tools handle readiness, the cardholder-data DLP (in simulation), verification, reporting, and scoping.

Script	Type	What it does
`CMPPCI-Discovery.ps1`	Read-only	Readiness + **role-separation** assessment: reads licensing SKUs (Graph), inventories the Compliance role groups and flags conflicting combinations (`PCI Roles – Separation Evidence`), checks whether **unified audit logging** is enabled (Req 10), confirms the **Credit Card Number** SIT (Req 3), and reports any existing Credit Card Number DLP policy/label and its mode.
`CMPPCI-Configuration.ps1`	`-WhatIf` default	**Cardholder-data DLP + action-plan** builder: with `-Commit`, creates `DLP-PCI-CardholderData` in **simulation (`TestWithNotifications`) mode** with the `DLP-PCI-CardholderData-Rule` matching the Credit Card Number SIT across the in-scope workloads, and writes the local `PCI Action Plan – Pilot` / `PCI Evidence – Register` tracking artifacts. Defaults to `-WhatIf`, printing exactly what it would create; makes **no enforcing** change.

Script	Type	What it does
`CMPPCI-Validation.ps1`	Read-only	**Technical-evidence** validator: confirms audit logging enabled + retention (Req 10), the DLP policy present and **in simulation mode** with the Credit Card Number rule (Req 3), the sensitivity label published, and role separation — the cross-check behind the assessment's improvement actions.
`CMPPCI-Report.ps1`	Read-only	**PCI posture** report: rolls up the control-to-evidence map, audit state, DLP mode + a simulation-activity summary from the unified audit log, role separation, and licensing to CSV for a leadership rollup; notes the portal Compliance Manager export as authoritative.
`CMPPCI-Set-Scope.ps1`	`-WhatIf` default	**Role/scope + enforcement** helper: previews and (with `-Commit`) applies role-group membership changes and — explicitly gated — moves `DLP-PCI-CardholderData` from **simulation to enforce** (`Set-DlpCompliancePolicy -Mode Enable`) so expansion/enforcement stays deliberate and reversible.

Named objects (verbatim — used across docs, scripts, and tests)

- **PCI-DSS assessment:** `PCI Assessment – Baseline – Pilot` → `PCI Assessment – Production`
- **Cardholder-data DLP (simulation-first):** policy `DLP-PCI-CardholderData`, rule `DLP-PCI-CardholderData-Rule` (Credit Card Number SIT)
- **Cardholder-data label:** `Label-PCI-CardholderData`
- **Tracking artifacts:** `PCI Action Plan – Pilot`, `PCI Evidence – Register`, `PCI Control Map – Record`, `PCI Scope – Record`, `PCI Posture – Report`, `PCI Roles – Separation Evidence`
- **Pilot scope group:** `SG-PCI-Pilot`

What this solution deliberately does NOT do

• It does **not** scope your **Cardholder Data Environment**, choose your **SAQ/version**, or decide whether the evidence **satisfies PCI-DSS** — those are **QSA / compliance / legal** determinations.

• It does **not** enforce the cardholder-data DLP until it has been **piloted in simulation** and validated; enforcement is an explicit, gated, reversible step.

• It does **not** implement the PCI-DSS requirements Purview can't cover — firewalls, segmentation, physical security, anti-malware, vulnerability/penetration testing — those are tracked as **evidence links** to their owning systems.

• It does **not** invent portal paths, cmdlets, or sensitive-information-type names — where authoring is portal-only (assessments, evidence, labels) the tools stop at readiness/validation and say so.

• It does **not** store or log real cardholder data; DLP validation uses **synthetic test card numbers** only.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CMPPCI-Configuration.ps1

Purpose: Stands up the PCI-DSS Requirement 3 cardholder-data control the safe way: with -Commit it creates the DLP policy DLP-PCI-CardholderData in SIMULATION (TestWithNotifications) mode with the rule DLP-PCI-CardholderData-Rule matching the built-in "Credit Card Number" sensitive information type across the in-scope workloads, and writes the local tracking artifacts PCI Action Plan - Pilot and PCI Evidence - Register from an owner-provided action CSV. It NEVER enforces (blocks) — enforcement is a separate, gated step in CMPPCI-Set-Scope.ps1 after validation. Defaults to -WhatIf, printing exactly what it would create/write. HOW TO RUN Dry run (default) — shows the intended DLP policy/rule and plan, changes nothing: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Configuration.ps1 -ActionCsv .\pci-actions.csv Apply — creates the DLP policy in SIMULATION and writes the tracking artifacts: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Configuration.ps1 -ActionCsv .\pci-actions.csv -Commit

How to run: Dry run (default) — shows the intended DLP policy/rule and plan, changes nothing: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Configuration.ps1 -ActionCsv .\pci-actions.csv Apply — creates the DLP policy in SIMULATION and writes the tracking artifacts: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Configuration.ps1 -ActionCsv .\pci-actions.csv -Commit

CMPPCI-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell (and optionally Microsoft Graph) and inventories the readiness for a Microsoft Purview PCI-DSS compliance program: the Compliance role groups and their membership (flagging CONFLICTING role combinations = the PCI-DSS Req 7 separation check), whether unified audit logging is enabled (Req 10), whether the built-in "Credit Card Number" sensitive information type is present (Req 3), and any existing Credit Card Number DLP policy / label and its mode. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPPCI-Discovery.ps1 Optionally include a Graph licensing check: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Discovery.ps1 -CheckLicensing

How to run: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Discovery.ps1 Optionally include a Graph licensing check: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Discovery.ps1 -CheckLicensing

CMPPCI-Report.ps1

Purpose: Read-only posture report for the PCI-DSS program. Rolls up the live control state for the report: unified-audit state + retention (Req 10), the cardholder-data DLP policy and its mode (Req 3), the cardholder-data label presence (Req 3), the Compliance Manager role-group membership / separation snapshot (Req 7), and the SKUs it can read (licensing). Exports CSV/HTML for the assessment package. It reports state; it changes nothing and it does not attest. HOW TO RUN Connect first: Connect-IPPSSession (and Connect-ExchangeOnline for the audit-config read). .\CMPPCI-Report.ps1 .\CMPPCI-Report.ps1 -CsvPath .\pci-posture.csv -HtmlPath .\pci-posture.html EXPECTED RESULTS PCI Posture – Report rows: audit ON + retention, DLP present + mode (should read simulation until enforcement is gated), label present, role separation snapshot, licensing. The portal Compliance Manager

export remains the authoritative score source. .NOTES White-label. Read-only. Whether the score/evidence SATISFIES PCI-DSS and how the CDE is scoped are QSA/compliance/legal determinations — DEFERRED. Compliance Manager scoring/export are portal operations. Feature availability is licensing-/tenant-dependent and evolving. No cmdlets invented. > [CmdletBinding()] param([string] $PolicyName = 'DLP-PCI-CardholderData', [string] $LabelName = 'Label-PCI-CardholderData', [string[]] $RoleGroups = @('ComplianceAdministrator','Compliance Manager Administrators','Compliance Manager Assessors','Compliance Manager Contributors','Compliance Manager Readers'), [string] $CsvPath, [string] $HtmlPath)

How to run: Connect first: Connect-IPPSSession (and Connect-ExchangeOnline for the audit-config read). .\CMPPCI-Report.ps1 .\CMPPCI-Report.ps1 -CsvPath .\pci-posture.csv -HtmlPath .\pci-posture.html EXPECTED RESULTS PCI Posture – Report rows: audit ON + retention, DLP present + mode (should read simulation until enforcement is gated), label present, role separation snapshot, licensing. The portal Compliance Manager export remains the authoritative score source. .NOTES White-label. Read-only. Whether the score/evidence SATISFIES PCI-DSS and how the CDE is scoped are QSA/compliance/legal determinations — DEFERRED. Compliance Manager scoring/export are portal operations. Feature availability is licensing-/tenant-dependent and evolving. No cmdlets invented. > [CmdletBinding()] param([string] $PolicyName = 'DLP-PCI-CardholderData', [string] $LabelName = 'Label-PCI-CardholderData', [string[]] $RoleGroups = @('ComplianceAdministrator','Compliance Manager Administrators','Compliance Manager Assessors','Compliance Manager Contributors','Compliance Manager Readers'), [string] $CsvPath, [string] $HtmlPath)

CMPPCI-Set-Scope.ps1

Purpose: The governed change tool for two high-impact PCI actions: (1) role-group membership for least-privilege separation — composes Add-RoleGroupMember / Remove-RoleGroupMember for the Compliance Manager role groups; (2) moving DLP-PCI-CardholderData from SIMULATION to ENFORCE (Set-DlpCompliancePolicy -Mode Enable), or back to simulation for rollback. SAFE BY DEFAULT: runs in -WhatIf and prints exactly what it would change; -Commit is required to apply, and the DLP enforce path additionally requires -ConfirmEnforce so it is never bundled with a membership edit. HOW TO RUN Connect first: Connect-IPPSSession (a role able to manage role groups / DLP, e.g. Compliance Admin). Role membership (dry-run then commit): .\CMPPCI-Set-Scope.ps1 -RoleGroup "ComplianceAdministrator" -Add user@contoso.com -WhatIf .\CMPPCI-Set-Scope.ps1 -RoleGroup "ComplianceAdministrator" -Remove old@contoso.com -Commit Enforce the validated DLP (deliberate, gated): .\CMPPCI-Set-Scope.ps1 -EnforceDlp -PolicyName "DLP-PCI-CardholderData" -WhatIf .\CMPPCI-Set-Scope.ps1 -EnforceDlp -PolicyName "DLP-PCI-CardholderData" -Commit -ConfirmEnforce Roll DLP back to simulation: .\CMPPCI-Set-Scope.ps1 -SimulateDlp -PolicyName "DLP-PCI-CardholderData" -Commit EXPECTED RESULTS -WhatIf: one line per intended change, nothing applied. -Commit: the membership change is applied, or (with -ConfirmEnforce) the DLP policy Mode moves to Enable. Enforcement is a separate step done only after 05-testing.md passes. .NOTES White-label. -WhatIf by default; enforcement double-gated (-Commit + -ConfirmEnforce). Security & Compliance PowerShell. No credentials in code. CDE scope / SAQ / attestation adequacy are QSA/ compliance determinations — this tool applies the approved change, it does not decide it. No cmdlets invented. > [CmdletBinding()] param(Role membership [string] $RoleGroup, [string[]] $Add, [string[]] $Remove,

How to run: Connect first: Connect-IPPSSession (a role able to manage role groups / DLP, e.g. Compliance Admin). Role membership (dry-run then commit): .\CMPPCI-Set-Scope.ps1 -RoleGroup "ComplianceAdministrator" -Add user@contoso.com -WhatIf .\CMPPCI-Set-Scope.ps1 -RoleGroup

"ComplianceAdministrator" -Remove old@contoso.com -Commit Enforce the validated DLP (deliberate, gated): .\CMPPCI-Set-Scope.ps1 -EnforceDlp -PolicyName "DLP-PCI-CardholderData" -WhatIf .\CMPPCI-Set-Scope.ps1 -EnforceDlp -PolicyName "DLP-PCI-CardholderData" -Commit -ConfirmEnforce Roll DLP back to simulation: .\CMPPCI-Set-Scope.ps1 -SimulateDlp -PolicyName "DLP-PCI-CardholderData" -Commit EXPECTED RESULTS -WhatIf: one line per intended change, nothing applied. -Commit: the membership change is applied, or (with -ConfirmEnforce) the DLP policy Mode moves to Enable. Enforcement is a separate step done only after 05-testing.md passes. .NOTES White-label. -WhatIf by default; enforcement double-gated (-Commit + -ConfirmEnforce). Security & Compliance PowerShell. No credentials in code. CDE scope / SAQ / attestation adequacy are QSA/ compliance determinations — this tool applies the approved change, it does not decide it. No cmdlets invented. > [CmdletBinding()] param(Role membership [string] $RoleGroup, [string[]] $Add, [string[]] $Remove,

CMPPCI-Validation.ps1

Purpose: Cross-checks that the technical controls the PCI-DSS assessment's improvement actions claim are actually in place: unified audit logging enabled + retention (Req 10); the cardholder-data DLP policy present and IN SIMULATION MODE with the Credit Card Number rule (Req 3); the sensitivity label published (Req 3); and least-privilege role separation (Req 7). This is the evidence cross-check — it confirms the controls exist and are in the expected state, so nothing is marked "implemented" without proof. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CMPPCI-Validation.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\CMPPCI-Validation.ps1